**1. Mixed Signal SoC**

A **mixed-signal System-on-Chip (SoC)** integrates both analog and digital circuits on a single chip.

- **Digital modules** are designed using RTL (Verilog/VHDL) and synthesised into logic gates
- **Analog macros** (DACs, PLLs, SRAMs, multiplexers) are manually laid out by analog designers
- Analog IP is treated as a **black box** during digital integration
- Integration requires **hierarchical abstraction** — analog macro described via LEF + LIB
- As process nodes shrink, the demand for tighter integration grows
- A **divide-and-conquer approach** is used: analog and digital are designed separately, then combined during the PnR stage

---

**2. RTL2GDS Flow**

The RTL-to-GDSII flow converts a Verilog hardware description into a fabrication-ready chip layout.

**Flow Steps**

Verilog RTL

↓ Yosys synthesis → gate-level netlist

Synthesis

↓ chip area, macro locations, power rings

Floorplanning

↓ IO pad placement

IO Placement

↓ global_placement_or → detailed_placement

Placement

↓ tap_decap_or

Tap/Decap Insertion

↓ gen_pdn

Power Distribution Network (PDN)

↓ run_routing (TritonRoute)

Routing

↓ run_magic_drc

DRC / LVS Verification

↓ run_magic

Final GDS Layout

**Key OpenLane Interactive Commands**

tcl

```
package require openlane 0.9
prep -design design_mux -overwrite

# Inject custom analog macro LEF
set lefs [glob $::env(DESIGN_DIR)/src/lef/*.lef]
add_lefs -src $lefs

run_synthesis
init_floorplan_or
place_io
global_placement_or
detailed_placement
tap_decap_or
detailed_placement
gen_pdn
run_routing
run_magic_drc
run_magic
```

**Why interactive?** Because a hardened analog macro requires manual LEF injection at a specific stage. The automated flow.tcl cannot handle this.

---

### 3. OpenROAD Project

- **OpenROAD** is a DARPA-funded open-source hardware compiler
- Translates RTL to GDS in **under 24 hours** with no human intervention
- Performs: placement, clock tree synthesis (CTS), routing, timing optimization, verification
- Used as the **core engine inside OpenLane**
- For mixed-signal designs, OpenROAD handles the **digital top-level hierarchy**
- Analog macro is plugged in via LEF/LIB

**More info:** The OpenROAD Project

---

### 4. OpenROAD / OpenLane Installation

**Recommended: Docker Installation**

bash

```
# Pull and run OpenLane Docker container
export PDK_ROOT=<path to skywater-pdk and open_pdks>
docker run -it -v $(pwd):/openLANE_flow \
    -v $PDK_ROOT:$PDK_ROOT \
    -e PDK_ROOT=$PDK_ROOT \
    -u $(id -u $USER):$(id -g $USER) \
    openlane:rc2

# Inside Docker — verify installation
./flow.tcl -design spm
```

**Setup New Design Project**

bash

```
cd designs
```

mkdir design_mux

# Generate default config

./flow.tcl -design design_mux -init_design_config

---

**5. Inputs Required for Physical Design**

| Input File | Format | Purpose |
|---|---|---|
| Top-level design | .v (Verilog) | Defines circuit functionality and hierarchy |
| Analog macro model | .v (Verilog) | Behavioral model for simulation |
| Physical abstraction | .lef | Pin locations, dimensions, routing blockages |
| Timing model | .lib (Liberty) | Timing arcs, setup/hold constraints |
| Technology file | .tech | Process-specific rules for Magic |
| Flow config | config.tcl | OpenLane configuration variables |

---

**6. Obtaining IP**

- The **AMUX2_3V** (2:1 analog multiplexer) source: avsdmux2x1_3v3
- Original layout was for **OSU018** technology
- Modified to work with **SKY130** using Magic layout tool + sky130A.tech
- Verilog behavioral model sourced from **Efabless PicoRV-32** repository:

bash

git clone https://github.com/efabless/raven-picorv32.git

**Tips for IP Design with SKY130**

- Macro height must be a **multiple of 2.72 µm** (SKY130 site height for fd_sc_hd)
    - Single height: 2.72 µm
    - Dual height: 5.44 µm ← used in this design
- Supply nets (VDD/VSS) must be **horizontal** to align with power rails
- For dual-height macro: need **3 supply nets** (VDD–VSS–VDD)
- The top-level cell should include only the relevant portion of the layout

---

### 7. Verilog Files

| File | Role |
|---|---|
| design_mux.v | Top-level: instantiates AMUX2_3V macro + SPI logic |
| AMUX2_3V.v | Behavioral model of the 2:1 analog multiplexer |
| raven_spi.v | SPI controller (from Efabless raven-picorv32) |
| spi_slave.v | SPI slave module |

**Important:** The Verilog files must NOT contain power/ground pins — OpenLane adds these automatically during PnR.

---

### 8. LEF File

The **Library Exchange Format (LEF)** file provides an abstract physical description of the analog macro for use by the PnR tool.

**What LEF Contains**

```
MACRO AMUX2_3V
 CLASS CORE ;
 ORIGIN 0.000 0.000 ;
 SIZE 10.920 BY 5.440 ;
 SITE unithddbl ;
 PIN A
  DIRECTION INPUT ;
  USE SIGNAL ;
  PORT
   LAYER met1 ;
    RECT ... ;
  END
 END A
 ...
```

OBS

LAYER met1 ;

RECT ... ;

END

END AMUX2_3V

**Getting LEF from Magic**

bash

magic -T ~/sky130A.tech ~/AMUX2_3V.mag

In Magic Tcl console:

tcl

lef write AMUX2_3V.lef

---

**9. LEF Limitations and Fixes**

**Problems Found in Auto-Generated LEF**

1. **Missing pin declarations** — only labels present, not proper ports
2. **Missing CLASS field** — PnR tool can't classify the macro
3. **Missing ORIGIN** — macro not aligned to (0,0)
4. **Missing SITE field** — no standard-cell site assignment
5. **Wrong HEIGHT** — not a multiple of 2.72 µm

**Fixes Applied in Magic Tcl Console**

**For Metal Layer Labels → Ports**

tcl

# Select the area with the label, then:

port make

# Verify:

port name

**For Polysilicon Layer Labels → Ports**

tcl

# 1. Enable grid

g

# 2. Erase label on poly

erase labels

# 3. Connect polycont to poly

# 4. Connect locali to polycont

# 5. Create new label on locali

label 'pin_name'

# 6. Convert to port

port make

**Set Required LEF Fields**

tcl

# Set CLASS

property LEFclass CORE

# Set ORIGIN to (0,0)

# First check current position:

box

# Move to origin:

move origin right <llx>um

move origin bottom -<lly>um

# Set SITE

property LEFsite unithddbl

# Set bounding box (height = 5.44 μm)

property FIXED_BBOX {0 24 874 568}

# Set pin direction and use

port class input    # for input pins

port class output   # for output pins

port class inout    # for power/ground

port use signal     # for signal pins

port use power      # for VDD

port use ground     # for GND

# Regenerate LEF

lef write AMUX2_3V.lef

**Check:** The final LEF should have an OBS section at the end. If not, the polysilicon pin conversion was not done correctly.

---

**10. LIB File (Liberty Timing Model)**

The LIB file describes timing behavior of the macro for static timing analysis.

**Generating LIB from Verilog**

bash

perl verilog_to_lib.pl AMUX2_3V AMUX2_3V

This generates AMUX2_3V.lib from AMUX2_3V.v.

**Note:** Pin names in the Verilog file must match exactly with the LEF. Rename pins in AMUX2_3V.v if they differ from the LEF pin names.

---

**11. OpenLane Flow — Configuration**

**config.tcl Key Settings**

tcl

set ::env(DESIGN_NAME) "design_mux"

set ::env(VERILOG_FILES) [glob $::env(DESIGN_DIR)/src/*.v]

set ::env(CLOCK_PORT) "clk"

set ::env(CLOCK_PERIOD) "10.0"

# Analog macro settings

set ::env(MACRO_PLACEMENT_CFG) $::env(DESIGN_DIR)/macro_placement.cfg

set ::env(PL_TARGET_DENSITY) 0.4

**Adding Input Files**

```
designs/design_mux/
├── src/
│   ├── design_mux.v
│   ├── AMUX2_3V.v
│   ├── raven_spi.v
│   ├── spi_slave.v
│   └── lef/
│       └── AMUX2_3V.lef
└── config.tcl
```

---

**12. Floorplanning**

- Determines **chip dimensions** and **macro locations**
- AMUX2_3V is placed as a hard macro at this stage
- Command: init_floorplan_or
- After floorplan, view in Magic:

bash

```
magic -T ~/sky130A.tech lef read ~/merged.lef def read design_mux.floorplan.def
```

---

**13. Placement**

- **Global placement:** global_placement_or — places cells approximately
- **Detailed placement:** detailed_placement — legalises positions
- **Tap/Decap insertion:** tap_decap_or — adds well tap and decap cells
- Run detailed placement again after tap/decap insertion

**Issue faced:** Macro height mismatch caused detailed placement to fail. The AMUX2_3V height was not exactly 5.44 µm.

---

**14. Routing**

- Command: run_routing — invokes **TritonRoute**
- Performs global and detailed routing
- View routed layout:

bash

```
magic -T ~/sky130A.tech lef read ~/merged.lef def read design_mux.def
```

---

**15. DRC Cleaning**

- Command: run_magic_drc
- Verifies layout satisfies SKY130 manufacturing rules
- View DRC results:

bash

```
magic -T ~/sky130A.tech design_mux.drc.mag
```

- All violations must be fixed before final GDS generation

---

**16. Final GDS Layout Generation**

- Command: run_magic
- Output: design_mux.mag and final GDSII file
- View final layout:

bash

```
magic -T ~/sky130A.tech design_mux.mag
```

---

**17. AI Workflow Summary**

| Stage | AI Tool Used | What Was Asked | Verified? |
| --- | --- | --- | --- |
| Repo understanding | Claude | Explain folder structure and flow | ✅ Yes |
| LEF decoding | Claude | Explain MACRO, PIN, TIMING sections | ✅ Yes |
| Tool setup | ChatGPT | Docker install commands for OpenLane | ✅ Yes |
| LEF fix | Claude | Magic Tcl commands to fix LEF | ✅ Yes |
| Error debug | ChatGPT | "Cannot find LEF macro" fix | ✅ Yes |
| Report | Claude | IEEE LaTeX two-column report | ✅ Yes |

---

### 18. Known Issues and Notes

- **RePlace update:** May need to update RePlace to the latest version for low-instance-count designs
- **Placement fix:** If placement still fails, comment out this block in scripts/openroad/or_replace.tcl:

tcl

```
# global_placement \
#   -density $::env(PL_TARGET_DENSITY) \
#   -verbose 3
```

- **Power pins in Verilog:** Remove all VDD/VSS port declarations from Verilog files before running synthesis
- **Macro LEF:** Add the LEF both in config.tcl AND in the interactive flow command (add_lefs)